PRODUCT VISION DOCUMENT

TravelMatch

What the platform does today, how to take it live on the internet, and a complete roadmap for turning it into a world-class travel product.

5	3	20+	4
CORE FEATURES	DEPLOYMENT OPTIONS	FUTURE FEATURES	ROADMAP PHASES

What It Does

Current platform capabilities and how each feature works for users

Going Live

Step-by-step guide to publishing on Railway, Render, and a custom domain

Improvements

Technical upgrades: real-time WebSockets, maps, AI matching, performance

Future Roadmap

4-phase plan from MVP polish to mobile app to monetization

CHAPTER 1

What TravelMatch Is & Does Today

A clear picture of every feature currently working in the platform and the real value it delivers to users.

> **TravelMatch connects solo travelers who share the same style, budget, and destinations — so they can plan, coordinate, and split costs together.**

The Problem It Solves

Solo travel is growing fast but it has a core problem: finding someone *compatible* to travel with is hard. Generic social platforms don't understand travel. Travel apps don't have a social layer. Group chats on WhatsApp don't track who paid for what. TravelMatch puts all of this in one place — finding the right person, planning the trip, and handling money fairly.

Five Live Features

1. Smart Compatibility Matching

When you sign up, you fill in a travel profile: your budget range, travel style, preferred destinations, trip length, and accommodation type. TravelMatch then runs a **weighted scoring algorithm** across every other user on the platform and ranks them by how well they match you.

💰

Budget (35%)

Most important factor. Economy, Mid-range, or Luxury. Adjacent tiers score 50%.

🗺️

Destinations (25%)

Overlap between your dream regions. Uses Jaccard similarity for fairness.

🧭

Style (20%)

Adventure, Cultural, Relaxation, or Mixed. "Mixed" scores well with everyone.

📅

Duration (12%)

Weekend, One Week, Two Weeks, or Extended trip preference.

🏨

Accommodation (8%)

Hostel, Airbnb, or Hotel. Exact match required for full score.

👁️

Like / Pass

Swipe-style cards. Passed users never appear again. Liked users are tracked.

2. Trip Group Management

Any user can create a trip group with a destination, dates, and description. Groups are private — joinable only with a **10-character invite code** that only the owner can share. The owner can add members by username, remove members, and the group can hold up to 12 people.

3. Real-Time Group Chat

Every group has a dedicated chat room. Messages appear live for all members using **AJAX polling** — the browser checks for new messages every 2 seconds without requiring a page refresh. Each message shows who sent it, the time, and is styled differently for your own messages vs others.

4. Collaborative Itinerary Planning

Groups can build a shared day-by-day plan. Any member can add days (with a title and date) and activities within each day (with times, location, notes, and an assignee). Activities can be **drag-and-reordered** by any member and the order is saved instantly.

5. Shared Expense Tracking & Settlement

Track every shared cost — who paid, how much, and how to split it. Three split modes are supported: **equal split** (everyone pays the same), **exact split** (specify each person's share), and **percentage split**. The platform then automatically calculates the **minimum number of bank transfers** needed to settle all debts.

✅ **What makes the settlement smart**

If 4 people have $150 in debts between them, a naive approach might require 6 transfers. TravelMatch's greedy algorithm settles it in at most 3 (n-1 transfers for n people). This is mathematically optimal.

Who Uses It & Why

USER TYPE	WHAT THEY USE	WHY IT HELPS
Solo traveler looking for company	Match feed, profile, like/pass	Finds compatible companions without awkward Facebook posts
Group trip organizer	Group creation, invite codes, itinerary	One app for planning — no juggling Google Docs + WhatsApp
Friend group on a trip	Chat, expenses, settlement	No more arguing about who owes who — auto-calculated
Frequent traveler	All features	Builds a network of trusted travel companions over time

Current Technical Scope

AREA	CURRENT IMPLEMENTATION
Backend	Django 4.2 (Python) — all logic server-side
Database	SQLite — single file, works locally, no setup needed
Real-time	AJAX polling every 2 seconds — works without WebSockets
Authentication	Django's built-in session auth — secure, cookie-based
Frontend	Bootstrap 5.3 + custom CSS + vanilla JavaScript
Hosting	Local only (python manage.py runserver)

CHAPTER 2

How to Publish the Website

A complete, step-by-step guide to taking TravelMatch from your laptop to the internet — free options included.

Right now TravelMatch runs only on your computer. To make it accessible at a real URL (like `travelmatch.com`) you need to deploy it to a server. Here's everything you need to do — from easiest to most professional.

ℹ️ **Before deploying: 4 things to change in settings.py**

These are required for any production deployment — they prevent security vulnerabilities.

```
# 1. Turn off debug mode (never expose error details publicly)
DEBUG = False

# 2. Set your domain (replace with your actual domain)
ALLOWED_HOSTS = ["travelmatch.railway.app", "www.yourdomain.com"]

# 3. Use a real secret key (never hardcode — read from environment variable)
import os
SECRET_KEY = os.environ.get("SECRET_KEY", "fallback-for-local-only")

# 4. Tell Django where to collect static files (CSS/JS) for production
STATIC_ROOT = BASE_DIR / "staticfiles"

# 5. Use PostgreSQL instead of SQLite in production
import dj_database_url
DATABASES = {"default": dj_database_url.config(default=os.environ.get("DATABASE_URL"))}
```

Option 1: Railway (Recommended — Free Tier Available)

Railway is the simplest way to deploy a Django app. You connect your GitHub repo and it builds and deploys automatically every time you push code.

1 **Install required packages**

These are needed for production deployment. Add them to `requirements.txt`.

```
# Run in terminal to install
pip install gunicorn whitenoise psycopg2-binary dj-database-url

# Then save to requirements.txt
pip freeze > requirements.txt
```

2 Create a Procfile (tells Railway how to run your app)

Create a file called `Procfile` (no extension) in your project root.

```
web: gunicorn travelmatch_project.wsgi --log-file -
```

3 Add WhiteNoise to serve static files (CSS/JS)

Edit `settings.py` to add WhiteNoise middleware — it serves your CSS/JS from the server without needing a separate CDN.

```
MIDDLEWARE = [
    "django.middleware.security.SecurityMiddleware",
    "whitenoise.middleware.WhiteNoiseMiddleware",  # ← add this second
    ...
]
STATICFILES_STORAGE = "whitenoise.storage.CompressedManifestStaticFilesStorage"
```

4 Push code to GitHub and connect to Railway

Create a free account at railway.app → New Project → Deploy from GitHub repo → Select your repo. Railway auto-detects it's a Python/Django app.

5 Add environment variables in Railway dashboard

Go to your project → Variables tab → add these:

VARIABLE	VALUE
`SECRET_KEY`	A long random string — use djecrety.ir to generate one
`DEBUG`	`False`
`ALLOWED_HOSTS`	`yourapp.railway.app`
`DATABASE_URL`	Auto-provided by Railway when you add a PostgreSQL plugin

6 Add PostgreSQL database plugin

In Railway: New → Database → Add PostgreSQL. Railway connects it automatically and sets `DATABASE_URL` for you.

7 Run migrations on the live server

In Railway dashboard → your service → Shell tab → run:

```
python manage.py migrate
python manage.py collectstatic --noinput
python manage.py createsuperuser
```

Your app is now live at **yourapp.railway.app** — share this URL with anyone!

Option 2: Render (Also Free)

Very similar to Railway. Go to render.com → New Web Service → Connect GitHub → Select repo. Set:

- **Build Command:** `pip install -r requirements.txt && python manage.py collectstatic --noinput`
- **Start Command:** `gunicorn travelmatch_project.wsgi`
- Add same environment variables as Railway
- Create a free PostgreSQL database in Render and connect it

Option 3: Custom Domain (e.g. travelmatch.com)

Once deployed on Railway or Render, you can point a custom domain at it.

1 Buy a domain name

Go to Namecheap, Google Domains, or GoDaddy. Search for your domain (e.g. `travelmatch.app`). Costs ~$10-15/year for a `.com` or `.app`.

2 Add the domain in Railway/Render dashboard

Railway: Settings → Domains → Add Custom Domain. It gives you a CNAME record to copy.

3 Update your domain's DNS settings

In Namecheap/GoDaddy → DNS Management → Add a CNAME record pointing your domain to the Railway/Render URL. DNS changes take 10 minutes to 24 hours to propagate.

4 HTTPS is automatic

Railway and Render both automatically provision a free SSL certificate (via Let's Encrypt) so your site runs on `https://` — no extra setup needed.

5 **Update ALLOWED_HOSTS**

Add your custom domain to the environment variable:

```
ALLOWED_HOSTS=travelmatch.app,www.travelmatch.app
```

⚠️ **Cost summary**

Railway free tier: $5/month credit (enough for a small app). Render free tier: spins down after 15 minutes of inactivity (slow first load). For a real production app with traffic, budget ~$7-15/month for hosting + ~$12/year for a domain = under $200/year total.

Deployment Comparison

PLATFORM	FREE TIER	EASE	PERFORMANCE	CUSTOM DOMAIN
Railway	$5/mo credit	⭐⭐⭐⭐⭐ Very easy	Good	✅ Yes (free SSL)
Render	✅ Free (sleeps)	⭐⭐⭐⭐ Easy	OK (slow cold start)	✅ Yes (free SSL)
Heroku	Paid only now	⭐⭐⭐⭐ Easy	Good	✅ Yes
DigitalOcean	$4/mo minimum	⭐⭐⭐ Medium	Excellent	✅ Yes
AWS / GCP	12-month trial	⭐⭐ Complex	Excellent	✅ Yes

CHAPTER 3

Technical Improvements

Concrete upgrades to make TravelMatch faster, more scalable, and production-ready — from quick wins to major overhauls.

Quick Wins (1–3 days each)

🗄️

Switch to PostgreSQL

SQLite is a single file — only one person can write to it at a time. PostgreSQL handles thousands of concurrent users. Required for any real deployment. Railway provides it free.

🔐

Email Verification

Add Django's `EmailMultiAlternatives` to send a verification link after signup. Prevents fake accounts. Use SendGrid (free tier: 100 emails/day).

🖼️

Profile Photos

Add an `ImageField` to Profile model. Store images on Cloudinary or AWS S3 — free for small volumes. Show real photos on match cards.

🔒

Password Reset via Email

Django has this built in — just connect a mail server. Users can recover locked accounts. Needs an SMTP service like SendGrid or Mailgun.

📱

Responsive Polish

Test on mobile screens. Fix the itinerary drag-drop for touch screens (SortableJS supports touch natively). Ensure chat input doesn't get hidden by mobile keyboard.

📊

Pagination

Match feed, group lists, and expense lists should be paginated. Without it, loading 1000 matches at once will be very slow. Django's `Paginator` class makes this easy.

Medium Improvements (1–2 weeks each)

Replace AJAX Polling with WebSockets (Real Real-Time Chat)

Currently chat polls every 2 seconds — meaning messages can arrive up to 2 seconds late, and every user sends a request every 2 seconds even when nothing is happening. **Django Channels** with Redis replaces this with a true persistent connection.

```
# Install Django Channels
pip install channels channels-redis

# Add to settings.py
INSTALLED_APPS += ["channels"]
ASGI_APPLICATION = "travelmatch_project.asgi.application"
CHANNEL_LAYERS = {
    "default": {
        "BACKEND": "channels_redis.core.RedisChannelLayer",
        "CONFIG": {"hosts": [("127.0.0.1", 6379)]},
    }
}
# Result: messages appear instantly, zero polling overhead
```

Google Maps / OpenStreetMap Integration

Currently `location_lat` and `location_lng` fields exist in `ItineraryActivity` but aren't used. Adding a map makes the itinerary far more useful:

- Auto-complete location search using Google Places API (or OpenStreetMap's Nominatim — free)
- Show activity pins on a map within the itinerary tab
- Calculate distances and travel times between activities

Search & Filter on Match Feed

Currently the feed shows all ranked matches. Add filters:

FILTER	IMPLEMENTATION
Minimum score threshold	Pass `threshold=0.6` to `get_ranked_matches()`
Filter by destination	Add a `destinations__contains` queryset filter
Filter by travel month	Check if months overlap before scoring
Search by username	Simple `username__icontains` filter

Notifications System

Add a `Notification` model and alert users when: someone likes them back, they're invited to a group, a new chat message arrives (if they're not in the chat), a new expense is added to their group.

Mutual Match Detection

Currently "like" just saves a record. Add logic: when user A likes user B *and* user B has already liked user A → it's a mutual match. Show a "You matched!" notification and suggest creating a trip group together.

Major Upgrades (1–4 weeks each)

🤖

AI-Enhanced Matching

Use OpenAI's API to analyze user bios and find deeper personality compatibility beyond just structured preferences. "Both enjoy slow travel and local food" vs "both like adventure sports".

📱

React Native Mobile App

A mobile app dramatically increases engagement. The Django backend already works as an API — add Django REST Framework and build iOS + Android with React Native or Flutter.

🌐

Social Login

Let users sign in with Google or Facebook using `django-allauth`. Removes signup friction — more users complete registration when they don't need a new password.

✈️

Flight & Hotel Search

Integrate Skyscanner or Amadeus API to search flights directly from a group's itinerary. Show real prices and let members vote on flight options.

🌍

Multi-Currency Expenses

Fetch live exchange rates (via Open Exchange Rates API — free tier). Convert all expenses to one base currency for accurate settlement calculations across mixed currencies.

📸

Trip Photo Gallery

Add a shared photo album per group. Members upload trip photos which are stored on Cloudinary. Creates lasting memories and social proof for the platform.

Performance & Scalability

ISSUE	SOLUTION	IMPACT
Matching scans all users on every page load	Cache results in Redis with 10-min TTL, invalidate on profile update	10–100x faster for large user bases
SQLite single-writer bottleneck	Switch to PostgreSQL	Handles thousands of concurrent writes
Chat polling = constant server load	Django Channels WebSockets	90% reduction in server requests
No image optimization	Cloudinary auto-resize + WebP conversion	Faster page loads on mobile
No caching layer	Add Redis + django-cacheops	Expensive DB queries cached
Synchronous views block under load	Migrate to async views (Django 4.2 supports this)	Handles more concurrent users

Security Hardening



Rate Limiting

Add `django-ratelimit` to the login and signup views. Prevents brute-force password attacks and account enumeration.



Content Security Policy

Add `django-csp` to set CSP headers. Prevents XSS attacks by whitelisting which scripts and styles can load on your pages.



Input Sanitization

All user text content (bios, messages, activity names) should be escaped in templates — Django does this by default with `{{ }}` — but double-check no `{{ |safe }}` is used.



Audit Logging

Log who created/deleted groups and expenses. Useful for disputes ("I didn't delete that expense"). Use `django-auditlog` — one line per model.

CHAPTER 4

Future Features & Product Vision

What TravelMatch could become — from a student project to a full travel social platform.

> **The global online travel market is worth $800 billion. TravelMatch targets the fastest-growing segment: solo and group independent travelers who plan their own trips.**

Feature Roadmap by Priority

✅ **User matching with compatibility algorithm** — Done
Weighted scoring across 5 travel preferences

✅ **Group trip management with invite codes** — Done

✅ **Real-time group chat (AJAX polling)** — Done

✅ **Collaborative drag-to-reorder itinerary** — Done

✅ **Expense tracking with smart settlement** — Done

➡ SOON **Email verification & password reset** — Up Next
Required for real-world use

➡ SOON **Profile photos with Cloudinary** — Up Next

➡ SOON **Mutual match notifications** — Up Next

Alert when two users both like each other

💡 **WebSocket chat (Django Channels)** — Soon

Replace polling with true real-time

💡 **Google Maps itinerary integration** — Soon

💡 **Social login (Google / Facebook)** — Soon

💡 **Multi-currency expense conversion** — Soon

✨ **AI bio matching (OpenAI API)** — Future

Analyze personality beyond structured preferences

✨ **React Native mobile app (iOS + Android)** — Future

✨ **Flight & hotel search integration** — Future

✨ **Trip photo album & shared memories** — Future

✨ **Traveler reputation & review system** — Future

✨ **Visa & entry requirements lookup** — Future

✨ **In-app messaging between matched users (before group)** — Future

✨ **AI-powered packing list generator** — Future

CHAPTER 5

Development Roadmap — 4 Phases

A realistic, phased plan from where the project is today to a fully monetized travel platform.

1

PHASE 1 · NOW → MONTH 2

Polish & Deploy MVP

Make the current platform production-ready and get it online so real users can try it. Focus is reliability and trust — users won't use an app that feels unfinished.

- Deploy to Railway
- PostgreSQL
- Email verification
- Password reset
- Profile photos
- Mobile responsive
- Custom domain
- Bug fixes

2

PHASE 2 · MONTH 3–5

Improve Social Features & Real-Time

Make the core matchmaking and group experience feel alive. This phase turns a functional tool into something people actually enjoy using daily and want to recommend.

- Mutual matches
- WebSocket chat
- Notifications
- Google Maps itinerary
- Social login
- Match filters
- User reviews
- Multi-currency

3

PHASE 3 · MONTH 6–10

Mobile App & Platform Expansion

Build a mobile app (most travel planning happens on phones), add AI features, and integrate real travel data. This is where TravelMatch transitions from "useful tool" to "travel companion app".

- React Native app (iOS + Android)
- Push notifications
- AI bio matching
- Flight search API
- Hotel search API
- Packing list AI
- Photo albums
- Offline mode

4

PHASE 4 · MONTH 11+

Monetization & Scale

Build revenue streams that make sense for the community — no annoying ads, no paywalling core features. Focus on partnerships and premium features that power users will pay for.

- Premium subscriptions
- Affiliate hotel/flight commissions
- Travel insurance partnership

Verified traveler badge | Community forums | Travel agency partnerships | B2B group travel tools

Monetization Strategy

The best travel apps succeed by adding real value — not by charging for things users expect for free. Here's a tiered approach:

🆓

Free Tier (Always)

Core matching, 1 active group, group chat, basic itinerary, expense splitting. Enough to get full value from the platform.

⭐

Premium ($5–9/month)

Unlimited groups, see who liked you, advanced filters, profile boost in search, detailed compatibility breakdown, ad-free.

✈️

Affiliate Revenue

When users book flights/hotels through TravelMatch's search, earn 2–5% commission. Zero friction — users get a good price, platform earns passively.

🤝

B2B: Group Travel

Sell group management tools to travel agencies and tour operators. White-label the expense splitting and itinerary planning features.

Competitive Landscape

FEATURE	TRAVELMATCH	HOSTELWORLD	TRIPIT	SPLITWISE	COUCHSURFING
Travel buddy matching	✓ Algorithmic	~ Forums	✗	✗	~ Manual
Group trip planning	✓	✗	✓	✗	✗
Expense splitting	✓	✗	✗	✓ Best-in-class	✗
Group chat	✓	✗	✗	✗	~ Limited
Itinerary builder	✓	✗	✓ Strong	✗	✗
All-in-one	✓ **Only one**	✗	✗	✗	✗

🎯 **The unique opportunity**

No existing product combines travel companion matching + group coordination + expense splitting in one place. Users currently need 3–4 apps (Tinder/Facebook for finding people, WhatsApp for chat, Google Docs for planning, Splitwise for money). TravelMatch can own this entire workflow.

CHAPTER 6

Next Steps Checklist

A practical, prioritized action list — what to do this week, this month, and this quarter.

This Week — Make It Deployable

▫ Install production deps

`pip install gunicorn whitenoise psycopg2-binary dj-database-url`
Update requirements.txt

▫ Create Procfile

Add `Procfile` to root: `web: gunicorn travelmatch_project.wsgi`

▫ Update settings.py

DEBUG=False, read SECRET_KEY from env, add STATIC_ROOT, add WhiteNoise middleware

▫ Push to GitHub

`git init` → `git add .` → `git commit` → create GitHub repo → `git push`

▫ Deploy to Railway

railway.app → New Project → Deploy from GitHub → add environment variables → add PostgreSQL plugin

▫ Run migrations live

Railway Shell: `python manage.py migrate` then `python manage.py createsuperuser`

This Month — Core Quality

SOON **Add email verification on signup** — prevents fake accounts, required for professional appearance

SOON **Add profile photo upload** — matching without photos has much lower engagement

SOON **Add mutual match detection** — "You and Alice both liked each other!" is a key engagement moment

SOON **Add pagination** — match feed, group list, expense list should load 20 items at a time

➡️ SOON **Test on mobile** — fix anything that's broken on phone screens, especially drag-drop and chat

➡️ SOON **Add a .gitignore** — make sure `db.sqlite3`, `.env`, `__pycache__`, `.venv` are excluded from git

This Quarter — Core Growth Features

💡 **Switch chat to WebSockets** — install Django Channels + Redis for true real-time messaging

💡 **Add Google Maps to itinerary** — drag activities onto a map, see the day's route

💡 **Social login** — install django-allauth, add "Continue with Google" button

💡 **Notifications bell** — in-app alerts for new likes, group invites, new expenses

💡 **Direct messaging** — let matched users message each other before creating a group

💡 **Buy a domain and set up custom email** — travelmatch.app or travelmatch.co with hello@travelmatch.app

---

Essential .gitignore (Create This Now)

```
# File: .gitignore (in project root)

# Database — never commit, contains all user data
db.sqlite3

# Virtual environment — too large, not needed in repo
.venv/
.venv-1/
venv/

# Environment variables — contains secrets
.env
.env.local
*.env

# Python cache files
__pycache__/
*.pyc
*.pyo

# Django collected static (generated, not source)
staticfiles/

# OS files
.DS_Store
Thumbs.db

# IDE files
.vscode/
.idea/
```

⚠️ **Never commit your SECRET_KEY or database to GitHub**

If your GitHub repository is public, anyone who can see it can steal your SECRET_KEY and forge session cookies, gaining access to any account on your site. The `.gitignore` above prevents this. Always store secrets in environment variables.

Summary

RIGHT NOW	AFTER DEPLOYMENT	IN 6 MONTHS
✅ Smart matching algorithm	🌐 Live on the internet	💬 True real-time chat
✅ Trip group management	🔐 Email verification	🗺️ Maps in itinerary
✅ Group chat (polling)	🖼️ Profile photos	📲 Mobile app (iOS/Android)
✅ Drag-and-drop itinerary	🤝 Mutual matches	🔔 Notifications
✅ Expense splitting + settlement	📱 Mobile-optimised	💬 Direct messaging
✅ Runs locally on your Mac	🗄️ PostgreSQL database	🤖 AI-enhanced matching

🚀 **You've built the hardest part**

The compatibility algorithm, the settlement engine, the real-time chat architecture, and the full database schema are already done. These are the genuinely difficult engineering problems. Everything in this roadmap — deployment, new features, mobile app — builds on this foundation. The core is solid.